

React / Angular / Vue

Frontend Development

Why Frameworks Exist

Hand-written DOM manipulation gets unmanageable as an interface grows — keeping dozens of interdependent pieces of UI in sync with changing data by hand is error-prone and repetitive. Component frameworks solve this by letting you describe what the UI should look like for a given state, and having the framework handle updating the actual DOM efficiently when that state changes. React, Angular, and Vue all solve this same core problem with different philosophies — React with explicit JavaScript and a virtual DOM, Angular with a full opinionated framework and TypeScript-first design, Vue with HTML-like templates and a gentler learning curve — but the component-driven, data-drives-UI mental model transfers across all three.

Component-Driven Architecture

In every one of these frameworks, a UI is built from small, focused, reusable components composed into a tree — a page is a component made of smaller components, which are made of smaller components still. Good component design keeps each piece focused on one responsibility (a button component shouldn't also manage a shopping cart's totals) and passes data in through defined inputs (props in React/Vue, `@Input()` in Angular) rather than reaching out to grab data from elsewhere, which keeps components predictable and reusable across different parts of an app.

Routing

Single-page applications (SPAs) load one HTML page and then swap out content in place as the user navigates, rather than requesting a fresh full page from the server on every click. A router (React Router, Angular's built-in Router, Vue Router) matches the current URL to a component to render, keeps the browser's back/forward buttons working correctly, and lets you pass data through the URL itself (route params like `/courses/:id`, or query strings). Understanding routing is what makes an SPA feel like a normal multi-page website while actually staying a single page under the hood.

State Management Across Frameworks

All three frameworks distinguish local component state (owned by one component, not shared) from state that needs sharing across distant parts of the app. React commonly uses Context or a library like Redux/Zustand for shared state; Vue has its own reactive state system and libraries like Pinia; Angular commonly uses services combined with RxJS Observables. Despite different APIs, the underlying question is always the same: which component actually owns this

piece of data, and how does a change to it get communicated to every other part of the UI that depends on it?

Hooks and Composition Patterns

Modern React relies heavily on hooks — functions like `useState`, `useEffect`, and `useContext` that let function components manage state and side effects without needing class components. `useEffect` in particular handles anything that needs to synchronize a component with something outside React (fetching data on mount, subscribing to an event, setting up a timer), and its dependency array controls exactly when that synchronization re-runs. Vue's Composition API (`setup()`, `ref()`, `computed()`) and Angular's signals follow a similar underlying idea: give developers small, composable functions to build behavior from, instead of relying purely on inheritance or lifecycle methods baked into a class.

Learn More

React Official Documentation

<https://react.dev/>

Vue.js Official Documentation

<https://vuejs.org/guide/introduction.html>

Angular Official Documentation

<https://angular.dev/>